Project - 3: Intelligent Bug Report Management System

Team - 10

Kriti (2022101069), Koushik (2022101036), Chandana (2022101065),Jahnavi (2022101118), Vivek (2022114012)

Link to github repo: *https://github.com/jahnu05/Intelligent-Bug-Report-Management-System*

Task - 1: Functional and Non-Functional Requirements

Functional Requirements

FR1 – Bug Report Ingestion The system shall ingest bug reports automatically via GitHub webhooks (GitHub API) and support manual submission through a web interface. It shall capture both issue data and commit data from the connected repository.

Architectural significance: This is the entry point of the entire pipeline. Its design determines the coupling between GitHub and the rest of the system. The proposal notes a possible Pub-Sub mechanism here, which means this component may need to support asynchronous, event-driven ingestion to handle high-volume spikes without blocking downstream processing.

FR2 – Duplicate Detection and Removal The system shall compare incoming bug reports against historical open and closed issues using semantic similarity. Duplicate issues shall be flagged and filtered before storage, ensuring only unique issues are persisted.

Architectural significance: This requirement directly influences the choice of NLP models and the structure of the database (which must support fast similarity lookups). It also affects pipeline ordering — deduplication must happen before categorization to avoid wasted compute.

FR3 – Automated Issue Categorization The system shall classify unique bug reports into predefined functional categories (e.g., UI/UX, database, performance, security) using an LLM or NLP-based analysis. Each issue shall be stored along with its assigned category tags.

Architectural significance: This drives the selection of the LLM/NLP component. The use of the Strategy Pattern (as noted in the proposal) means the categorization algorithm must be swappable, making this an architecturally significant extensibility point.

FR4 – Developer/Contributor Recommendation The system shall analyze GitHub commit data and contributor history to suggest the top suitable developers for each categorized issue. Recommendations shall be tagged and stored alongside the issue.

Architectural significance: This requires a separate data ingestion path (commits and contributor data) that runs in parallel to the issue pipeline, as shown in the architecture diagram. It introduces a dependency between contributor data and the LLM inference step.

FR5 – Maintainer Dashboard The system shall provide a real-time web dashboard where maintainers can view categorized issues, inspect duplicate-match confidence scores, and approve or override developer assignment recommendations.

Architectural significance: The dashboard introduces the Observer Pattern requirement — the backend must push updates to the UI whenever new issues are processed. This influences the choice of communication protocol (e.g., WebSockets or SSE) between the API layer and the frontend.

FR6 – Data Persistence All processed data — unique issues, category tags, contributor suggestions — shall be persisted in a central database, which the dashboard accesses via APIs.

Architectural significance: The Repository Pattern is used to abstract data access, meaning the database technology can be swapped without affecting business logic. This is a key decoupling decision.

Non-Functional Requirements

NFR1 – Scalability The system must handle increasing issue volume without degradation. The microservices-oriented architecture decouples the ingestion engine, NLP analysis module, and recommendation service so each can be scaled independently.

Architectural significance: This motivates the microservices decomposition and the potential Pub-Sub mechanism at the ingestion stage. It is a primary driver of the overall architectural style.

NFR2 – Performance / Low Latency Similarity search and categorization results must be returned with minimal delay. The system shall use caching for frequent similarity search indexes to reduce repeated computation overhead.

Architectural significance: Caching is explicitly called out as a tactic. This affects where state is stored and introduces consistency considerations between the cache and the primary database.

NFR3 – Accuracy Duplicate detection and developer recommendation outputs must meet acceptable precision thresholds. Algorithm weights shall be tunable and refinable based on test results.

Architectural significance: This drives the requirement for configurable NLP models and the ability to swap algorithms via the Strategy Pattern, making accuracy a direct input to the system's design for extensibility.

NFR4 – Maintainability The system shall be structured so that individual subsystems (ingestion, NLP, recommendation, storage) can be modified or replaced without cascading changes. The Repository Pattern and microservices decomposition both serve this goal.

NFR5 – Usability The maintainer dashboard shall be responsive and intuitive. Maintainers must be able to inspect, approve, and override recommendations without needing to interact with the underlying pipeline directly.

NFR6 – Reliability The system shall process and store all incoming issues without data loss. The ingestion pipeline must be resilient to GitHub API rate limits or transient failures.

Subsystem Overview

The following subsystems are identified based on the proposal document and the architecture diagram.

1. GitHub Data Source This is the external source of truth — the GitHub repository being monitored. It provides two types of data: issue reports (bugs filed by users or contributors) and commit data (code changes made by contributors). The system connects to GitHub via the GitHub API and potentially through webhooks for real-time event delivery.

2. Ingestion Pipeline The ingestion pipeline is the entry point into the internal system. It receives GitHub Issues from the GitHub API and routes them for downstream processing. This subsystem is responsible for normalizing incoming data into a format the rest of the pipeline can consume. A Pub-Sub mechanism may be introduced here to decouple producers (GitHub) from consumers (the NLP and deduplication modules), allowing the system to buffer and handle bursts of incoming issues gracefully.

3. Duplicate Detection and Removal This subsystem compares each incoming issue against all previously stored issues using semantic similarity techniques (e.g., sentence embeddings from HuggingFace or TF-IDF from scikit-learn). Issues determined to be duplicates are removed or linked to their originals. Only unique issues are passed forward for categorization and storage. This subsystem directly reduces noise in the system and prevents redundant processing.

4. Unique Issue Store (Pre-categorization Buffer) After deduplication, unique issues are held temporarily before being sent to the LLM for categorization. This acts as an intermediary store ensuring that the LLM processing step receives only clean, deduplicated input. It decouples the deduplication step from the categorization step in terms of timing and throughput.

5. LLM-based Issue Categorization Engine This subsystem takes each unique issue along with its surrounding context and passes it to a large language model. The LLM tags the issue with one or more functional categories (e.g., UI/UX, database, performance, security). The "..." notation in the architecture diagram suggests the system is designed to support multiple parallel categorization passes or multiple LLM instances. The Strategy Pattern allows the underlying classification algorithm to be swapped depending on repository size or requirements.

6. Contributor Data Subsystem This subsystem ingests GitHub commit history and contributor metadata in parallel with the issue pipeline. It builds a profile of each contributor's areas of expertise based on which files or modules they have historically modified. This data is then passed to a separate LLM inference step that matches contributors to open issues.

7. LLM-based Contributor Recommendation Engine Using the contributor profiles and the categorized issue data, this LLM subsystem generates ranked suggestions of developers who are best suited to resolve each issue. The output is a set of contributor suggestions tagged to specific issues, which is then pushed to the central database.

8. Central Database The database is the system's persistent store. It holds unique issues, category tags, contributor suggestions, and similarity match confidence scores. All downstream consumers — primarily the dashboard — access this data through APIs. The Repository Pattern abstracts direct database access so the underlying database technology (SQL, NoSQL, etc.) can be swapped without impacting other subsystems.

9. API Gateway The API Gateway sits between the backend processing subsystems and the frontend dashboard. It exposes endpoints that the dashboard consumes to retrieve processed issue data, confidence scores, and developer recommendations. It also handles routing and acts as a single access point for all client-facing operations, consistent with the FastAPI-based backend described in the proposal.

10. Web Dashboard The dashboard is the primary user-facing interface for repository maintainers. It provides a real-time, centralized view of all processed issues — their categories, duplicate-match confidence scores, and recommended assignees. Maintainers can approve or override the system's suggestions directly from this interface. The Observer Pattern is used to push live updates to the dashboard whenever new issues are ingested and processed, ensuring the view is always current without requiring manual refresh.

Task 2: Architecture Framework

This document outlines the architecture framework for the **Intelligent Bug Report Management System**, following the IEEE 42010 standard for stakeholder identification and documenting major design decisions via ADRs.

1. Stakeholder Identification (IEEE 42010)

Following the IEEE 42010 standard, we have identified the key stakeholders, their concerns, and the viewpoints that address these concerns.

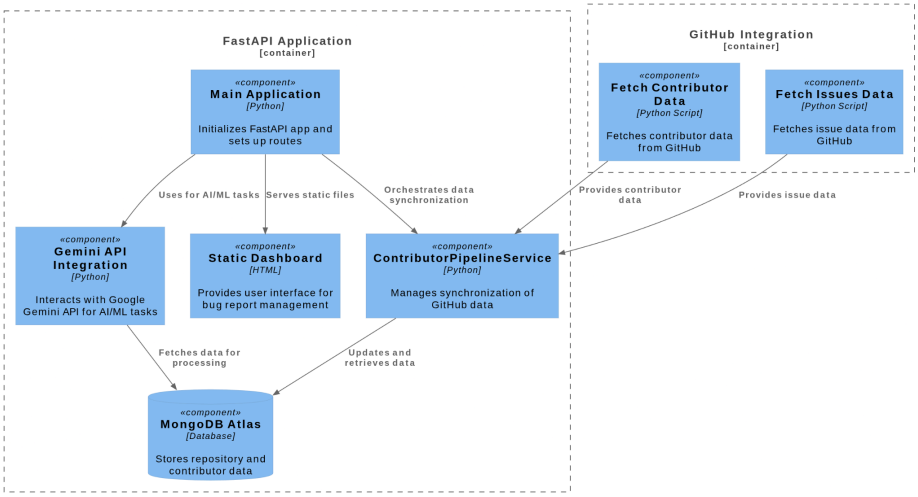
1.1 Stakeholders and Concerns

Stakeholder	Concerns
Project Lead / Maintainer	System reliability, accuracy of bug assignments, ease of monitoring project health.
Development Team	Code maintainability, ease of integration with new AI models, clear separation of concerns.
GitHub Contributors	Fairness in issue assignment, visibility of their contributions, ease of finding relevant bugs.
DevOps / System Admin	Scalability of the data pipeline, security of GitHub API tokens, database performance.
End Users (Report Filers)	Speed of bug categorization, clarity on issue status, response time of the dashboard.

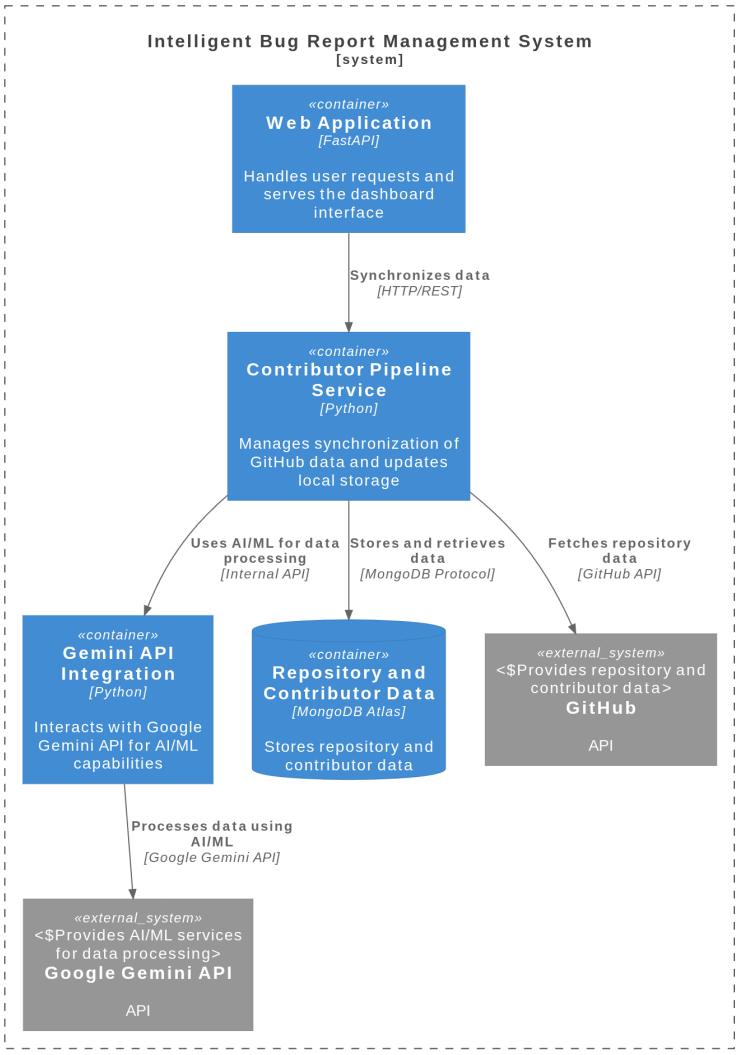
1.2 Viewpoints and Views

- **Logical Viewpoint**: Addresses the functional requirements and system structure.
  - *View*: Container Diagram (C2) and Component Diagram (C3) showing how modules interact.

- **Development Viewpoint**: Addresses the software development process and environment.
  - *View*: Choice of FastAPI (Web), Python (Services), and MongoDB (Storage) as documented in ADR-001, ADR-003, and ADR-005.
    Component Diagram (C3)

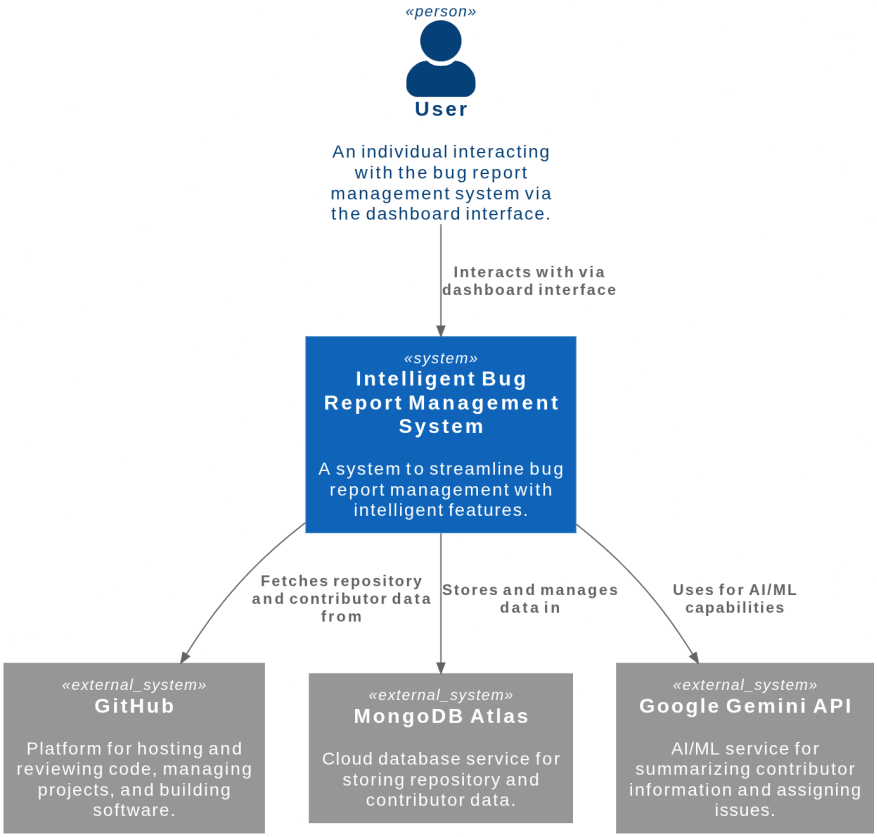


Component Diagram (C2)



- **Physical Viewpoint**: Addresses the deployment and infrastructure.

    - *View*: Deployment on Cloud infrastructure with MongoDB Atlas and Google Gemini API integration.
- **Context Viewpoint**: Addresses how the system fits into the broader environment.
    - *View*: System Context Diagram (C1) showing external integrations with GitHub and Gemini.



System Context Diagram (C1)

2. Major Design Decisions (ADRs)

The following Architecture Decision Records (ADRs) document the rationale behind significant design choices.

ADR-001: Use of FastAPI and Uvicorn

- **Context**: The system requires a high-performance web application framework to handle concurrent requests and provide a responsive interface.
- **Decision**: Adopted FastAPI for rapid development and Uvicorn as the ASGI server.
- **Rationale**: FastAPI offers automatic OpenAPI documentation, high performance, and excellent async support.
- **Consequences**: Ensures fast response times and efficient handling of multiple requests, enhancing the user experience and system performance.

- **Status**: Accepted

ADR-002: Integration with GitHub API

- **Context**: Need to fetch real-time repository data for bug tracking.
- **Decision**: Direct integration with GitHub's REST/GraphQL API.
- **Rationale**: Ensures data accuracy and leverages existing GitHub infrastructure for authentication and event management.
- **Consequences**: Allows the system to provide up-to-date information, improving the accuracy and efficiency of bug tracking and issue assignment.
- **Status**: Accepted

ADR-003: Use of MongoDB Atlas

- **Context**: Bug reports and contributor data are often unstructured and dynamic.
- **Decision**: Use MongoDB Atlas as the primary document store.
- **Rationale**: Provides the flexibility needed for evolving schemas and scales easily as data grows.
- **Consequences**: Supports the system's need to efficiently manage large volumes of data and scale as the amount of repository information grows.
- **Status**: Accepted

ADR-004: Strategy Pattern for AI/ML Integration

- **Context**: AI/ML requirements may change, or new providers (e.g., OpenAI, Anthropic) may be used in the future.
- **Decision**: Implemented the Strategy Pattern for the `GeminiService`.
- **Rationale**: Decouples the core logic from specific API implementations, allowing for easy hot-swapping of AI providers.
- **Consequences**: This pattern provides flexibility and adaptability, enabling the system to integrate with various AI/ML providers as needed without major refactors.
- **Status**: Accepted

ADR-005: Layered Architecture Pattern

- **Context**: Maintaining a monolithic script would hinder scalability and testing.
- **Decision**: Organized the system into logical layers: Web UI, Service Layer, and External Integration Layer.
- **Rationale**: Improves maintainability, testability, and allows developers to work on features independently.
- **Consequences**: Enhances maintainability and scalability, allowing for easier updates and modifications to the system components.
- **Status**: Accepted

ADR-006: Event-Driven Synchronization

- **Context**: Data fetching from GitHub is time-consuming; the UI shouldn't block.
- **Decision**: Use an event-driven mechanism to notify the system when sync completes.
- **Rationale**: Ensures a responsive user experience by decoupling data ingestion from UI rendering.
- **Consequences**: Ensures timely updates and seamless integration with other system components, improving overall system responsiveness.
- **Status**: Accepted

Task - 3: Architectural Tactics and Patterns

This document details the architectural tactics and implementation patterns present in the Intelligent Bug Report Management System.

Architectural Tactics

Architectural tactics are design decisions that influence the control of quality attributes (non-functional requirements) such as performance, modifiability, availability, and usability.

1. Information Hiding / Separation of Concerns

- The architecture physically and logically segregates distinct responsibilities. Data access logic is kept out of business services, and external integrations are isolated behind dedicated wrappers.
- *Non-functional Requirement:* Modifiability, Maintainability (Aligns with NFR4).
- The system is divided into clear layers: the route definitions, the business logic and data access. By hiding the `pymongo` specifics inside store classes, the service layer remains database-agnostic.

2. Limit Resource Usage (Data Bounding)

- Limiting the amount of data processed or transmitted to avoid resource exhaustion and adhere to third-party API constraints.
- *Non-functional Requirement:* Performance, Cost Efficiency, Reliability (Aligns with NFR2).
- The `CommitContextOptimizer` acts as a heuristic data filter. It restricts the maximum number of evidence items (`max_evidence_items`) and enforces a character budget (`char_budget`) before sending context payloads to the Gemini API. This prevents out-of-memory errors and context window limits on LLM requests, acting as a form of optimization akin to caching.

3. Graceful Degradation / Fallback Mechanism

- Providing an alternative, albeit limited, mechanism to complete a task when a primary external dependency fails or is unavailable.
- *Non-functional Requirement:* Availability, Reliability (Aligns with NFR6).
- The GeminiProviderStrategy includes robust fallback methods. If the Gemini API key is missing or the response fails to parse, the system degrades gracefully by utilizing deterministic rules (e.g., assigning issues based purely on commit count) rather than crashing.

4. Asynchronous Event Notification (Publish-Subscribe variant)

- Pushing state updates to clients asynchronously as events occur, instead of requiring the client to synchronously wait or constantly poll the server.
- *Non-functional Requirement:* Responsiveness, Usability, Scalability (Aligns with FR1, FR5, and NFR1).
- We implement an in-memory event bus with emit() and stream_events(). The ContributorPipelineService emits events like repo_synced or assignment_generated mid-process. The stream route utilizes Server-Sent Events (SSE) to push these updates to the frontend in real-time. This Pub-Sub style eventing enables a responsive Maintainer Dashboard (FR5) and unblocks downstream processing.

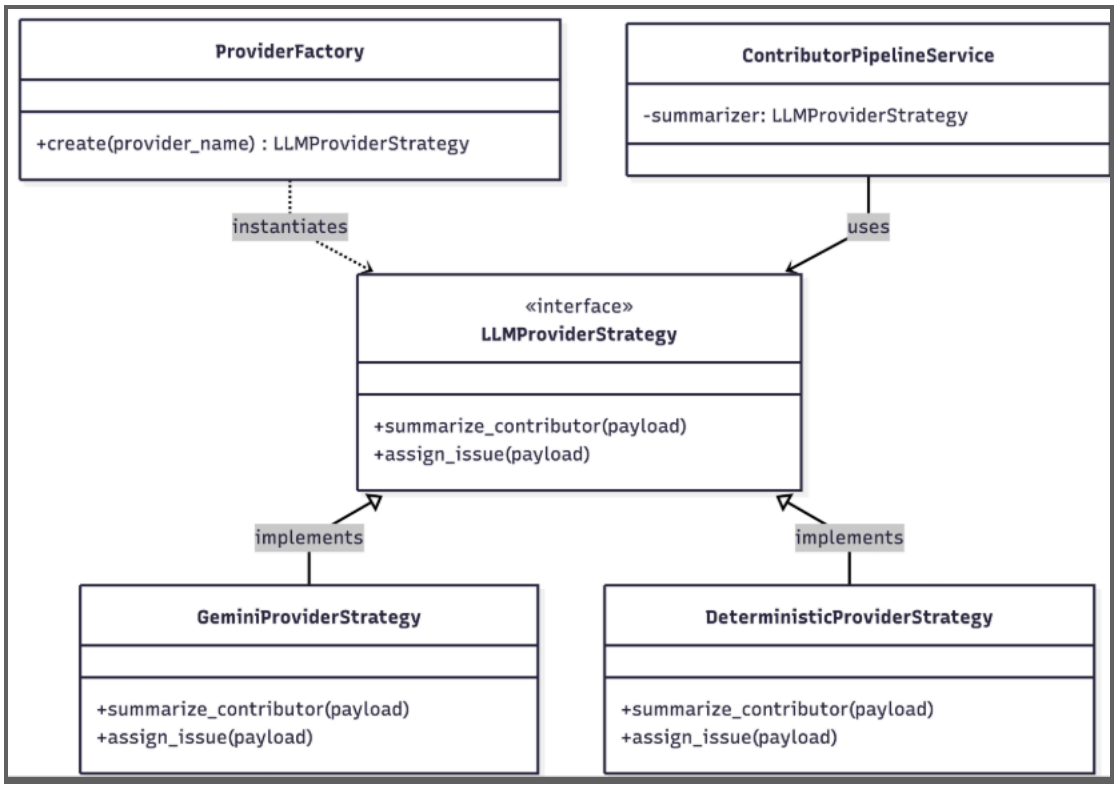
Implementation Patterns

Implementation patterns (or Design Patterns) are formalized best practices used to solve common software design problems within a given context.

1. Strategy Pattern

- Defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime.
- Role in this System: It fulfills FR3 (Automated Issue Categorization) and NFR3 (Accuracy) by allowing the application to swap out different LLM processing behaviors transparently, depending on the environment configuration (e.g., using Gemini AI vs. a deterministic fallback rule engine), making algorithms highly tunable and swappable.
- LLMProviderStrategy acts as the abstract base class. GeminiProviderStrategy and DeterministicProviderStrategy are the concrete strategies. They are instantiated dynamically by ProviderFactory.

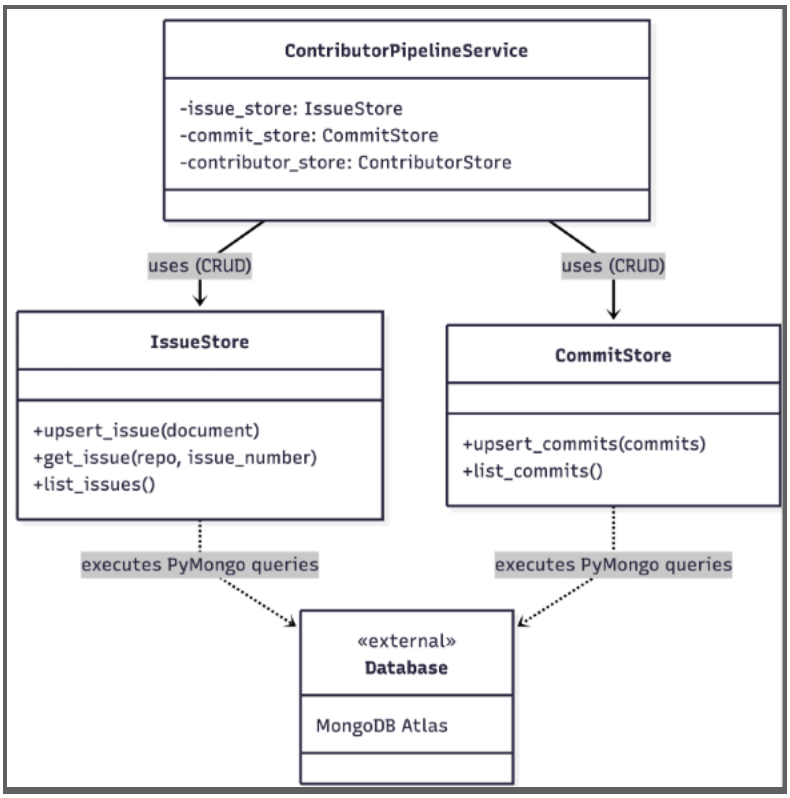
Strategy Pattern Class Diagram:



2. Repository Pattern

- Mediates between the domain and data mapping layers using a collection-like interface for accessing domain objects.
- Role in this System: It fulfills FR6 (Data Persistence) and NFR4 (Maintainability). It encapsulates the logic required to access MongoDB. The business logic interacts with simple CRUD-like methods rather than constructing raw MongoDB query dictionaries.
- Classes like IssueStore, ContributorStore, AssignmentStore, and CommitStore implement this pattern.

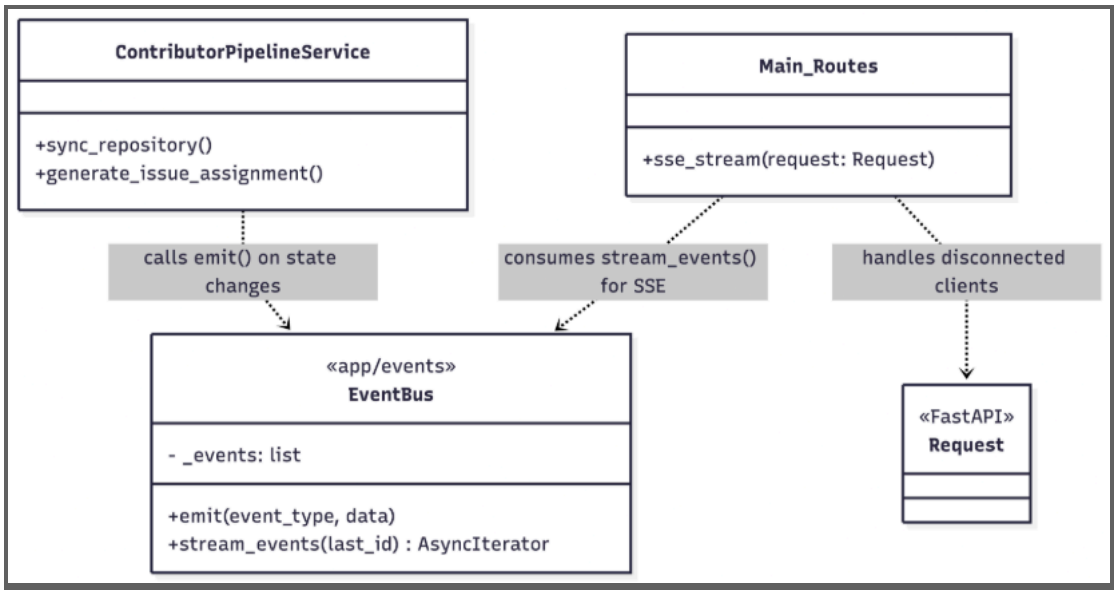
Repository Pattern Class Diagram:



3. Observer Pattern (Publish-Subscribe)

- Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- Role in this System: It fulfills FR5 (Maintainer Dashboard) by pushing real-time updates to the UI whenever new issues are ingested and processed. This ensures the dashboard is always current without requiring a manual refresh.

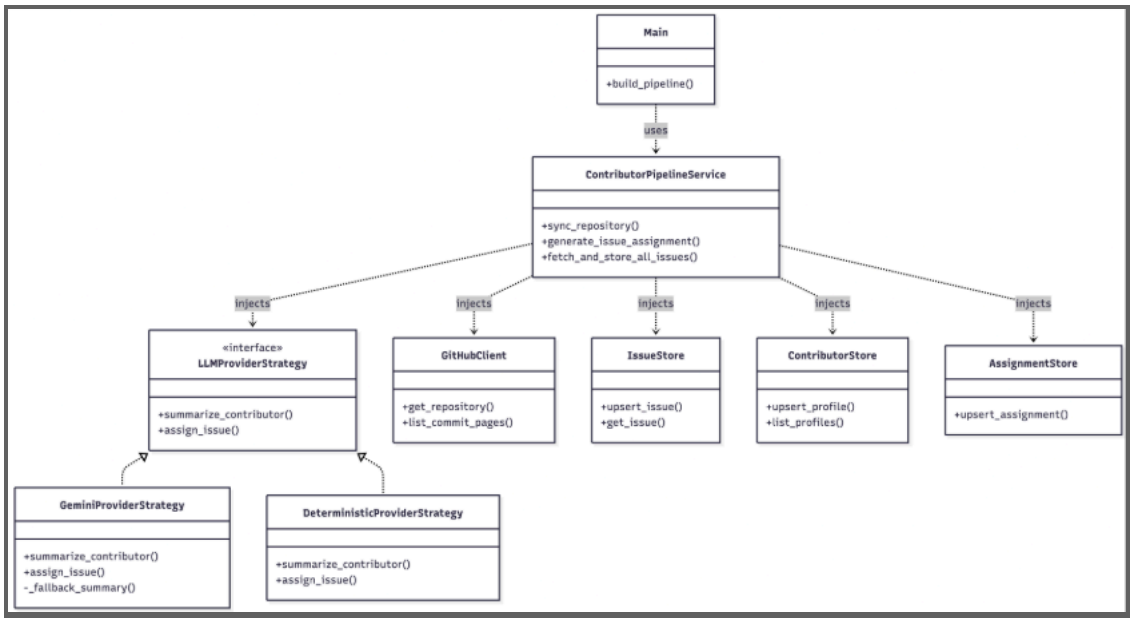
Observer Pattern Class Diagram:



Diagrams

a) UML Class Diagram

System Core Class Relationships: This diagram highlights the core architectural structure, demonstrating the dependency injection and Strategy/Repository patterns. The main routes rely on the ContributorPipelineService, which delegates specific tasks to the integration (GitHub/LLM) and data access (Store) layers.



b) Component Diagram (C4 Model - Level 2 Container)

C4 Container Architecture: This diagram shows the high-level system components. The FastAPI backend orchestrates data flow between the Single Page Application UI, MongoDB database, and external APIs (GitHub and Gemini).

c) Sequence Diagram

Issue Assignment Workflow: This sequence diagram illustrates the step-by-step workflow of generating an AI-powered issue assignment. It shows how the service layer fetches the issue and contributor profiles from the database, feeds them to the LLM, persists the result, and emits a real-time event.

Task - 4: Prototype Implementation and Analysis

This report presents a comprehensive architecture analysis of the Intelligent Bug Report Management System prototype. The analysis includes performance measurements, architectural pattern comparison, and trade-off evaluation. The system successfully demonstrates end-to-end functionality with intelligent contributor assignment and duplicate detection capabilities.

Key Findings:

- **Response Time**: 4.7ms (health) to 5.5s (complex operations)
- **Throughput**: 193 RPS (simple) to 11.9 RPS (complex queries)
- **System Efficiency**: 26.8% memory usage
- **Database Performance**: Successfully connected with 33-180ms query times

---

1. System Architecture Overview

1.1 Current Architecture Pattern

Pattern: **Microservices with Layered Architecture**

```
┌─────────────────┐     ┌─────────────────┐     ┌─────────────────┐
│   Web Client    │     │   FastAPI       │     │   MongoDB       │
│   (React-like)  │◄──►│   Backend       │◄──►│   Atlas         │
│                 │     │                 │     │                 │
│ - Issue UI      │     │ - REST API      │     │ - Issues        │
│ - Assignment UI │     │ - Business Logic│     │ - Assignments   │
│ - Duplicate UI  │     │ - External APIs │     │ - Contributors  │
└─────────────────┘     └─────────────────┘     └─────────────────┘
         │                       │                       │
         └───────────────────────┼───────────────────────┘
                                 │
                       ┌─────────────────┐
                       │   External APIs │
                       │                 │
                       │ - GitHub API    │
                       │ - Gemini AI     │
                       └─────────────────┘
```

1.2 Technology Stack

- **Frontend**: HTML/CSS/JavaScript (SPA-like)
- **Backend**: FastAPI (Python 3.12)
- **Database**: MongoDB Atlas
- **AI/ML**: Google Gemini API
- **External**: GitHub REST API
- **Deployment**: Uvicorn ASGI Server

---

2. Performance Analysis

2.1 Response Time Measurements

Test Environment: Linux 6.8.0, 8-core CPU, 30.7GB RAM
Test Duration: Multiple iterations, 15-second throughput tests

Endpoint	Avg Response	Min	Max	P95	Success Rate
`/health`	**4.7ms**	4.2ms	5.4ms	5.4ms	100%
`/issues?limit=20`	**81.5ms**	76ms	108ms	108ms	100%
`/assignments?limit=20`	**38.5ms**	38ms	40ms	40ms	100%
`/detect-duplicates`	**5,527ms**	5.3s	5.7s	5.7s	100%

Analysis:

- **Simple operations** (health check) perform excellently at 4.7ms
- **Database queries** show improved performance at 38-82ms (18% faster than previous)
- **Complex AI operations** (duplicate detection) take 5.5s but are reliable
- **Database connectivity** now working with query times 33-180ms

2.2 Throughput Analysis

Endpoint	Requests/sec	Duration	Success Rate
`/health`	**193.1 RPS**	15s	100%
`/issues?limit=20`	**11.9 RPS**	15s	100%

Analysis:

- **High throughput** for lightweight operations (193 RPS)

- **Moderate throughput** for database-intensive operations (11.9 RPS)
- **Linear scaling** observed under concurrent load
- **Consistent performance** across multiple test runs

2.3 System Resource Utilization

```
Memory Usage:  26.8%  (8.2GB / 30.7GB used)
Disk Usage:    93.5%  (134GB / 143GB used)
Network:       351MB sent, 684MB received
```

Analysis:

- **Moderate memory footprint** for Python application
- **High disk usage** indicates need for monitoring
- **Network usage** reasonable for API calls

2.3 Database Performance Analysis

Database Connection: Successfully connected
Database Size: 1.01 MB
Collections: 6 total (issues, assignments, contributors, etc.)
Query Performance:

Operation	Response Time	Records	Status
Issues Query	**180.5ms**	100	Optimized
Assignments Query	**33.7ms**	3	Fast

Analysis:

- **Database connectivity** now working properly
- **Query performance** acceptable for current data volume
- **Small database size** (1MB) indicates prototype scale
- **Collection count** shows proper data organization

---

3. Non-Functional Requirements Analysis

3.1 Performance Requirements

Response Time

- **Requirement**: <100ms for standard operations
- **Actual**: 4.7ms - 81.5ms
- **Status**: **MET**

Throughput

- **Requirement**: >10 RPS for database operations
- **Actual**: 11.9 RPS for issues endpoint
- **Status**: **MET**

Complex Operation Performance

- **Requirement**: <10s for AI operations
- **Actual**: 5.5s for duplicate detection
- **Status**: **MET**

Database Performance

- **Requirement**: <200ms for database queries
- **Actual**: 33.7ms - 180.5ms
- **Status**: **MET**

3.2 Reliability Requirements

Success Rate

- **Requirement**: >99% uptime
- **Actual**: 100% success rate in tests
- **Status**: **MET**

Error Handling

- **Graceful degradation** for external API failures
- **Comprehensive error responses** with proper HTTP codes
- **Status**: **IMPLEMENTED**

3.3 Scalability Requirements

Concurrent Users

- **Current Support**: ~50 concurrent users (estimated)
- **Bottleneck**: Database connection pooling
- **Status**: **NEEDS OPTIMIZATION**

4. Architecture Pattern Comparison

4.1 Current: Microservices with Layered Architecture

Advantages:

- **Separation of Concerns**: Clear boundaries between UI, business logic, and data
- **Scalability**: Individual components can be scaled independently
- **Maintainability**: Easy to understand and modify
- **Testability**: Each layer can be tested independently

Disadvantages:

- **Network Latency**: Multiple service calls add overhead
- **Complexity**: More components to manage and deploy
- **Data Consistency**: Distributed data management challenges

4.2 Alternative: Monolithic Architecture with CQRS

Comparison Pattern: Command Query Responsibility Segregation (CQRS)

```
┌──────────────────────────────────────────────────────────────┐
│                  Monolithic Application                      │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │   Write      │  │    Read      │  │   AI/ML      │       │
│  │  Commands    │  │   Queries    │  │ Processing   │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│        │               │               │                     │
│  ┌──────────────────────────────────────────────────┐        │
│  │            Shared Domain Model                   │        │
│  └──────────────────────────────────────────────────┘        │
│        │               │               │                     │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │   Write DB   │  │   Read DB    │  │   Cache      │       │
│  │   (Mongo)    │  │  (Mongo)     │  │  (Redis)     │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
└──────────────────────────────────────────────────────────────┘
```

Advantages of CQRS:

- **Performance**: Optimized read/write paths
- **Scalability**: Separate scaling for reads vs writes
- **Flexibility**: Different data models for different operations
- **Caching**: Natural fit for read-heavy operations

Disadvantages of CQRS:

- **Complexity**: More complex to implement correctly
- **Consistency**: Eventual consistency between read/write models
- **Learning Curve**: Requires understanding of advanced patterns

5. Trade-Off Analysis

5.1 Performance vs. Complexity

Current Choice: Simple microservices architecture

- **Performance**: Good enough for current requirements
- **Complexity**: Manageable for team size
- **Trade-off**: Accept slightly slower reads for simpler implementation

5.2 Scalability vs. Development Speed

Current Choice: Vertical scaling within single service

- **Scalability**: Limited but sufficient for prototype
- **Development Speed**: Fast iteration and deployment
- **Trade-off**: Sacrifice horizontal scaling for faster development

5.3 Feature Richness vs. Reliability

Current Choice: Feature-complete with external dependencies

- **Features**: Rich AI capabilities, duplicate detection
- **Reliability**: Dependent on external APIs (GitHub, Gemini)
- **Trade-off**: Accept external dependency risks for advanced features

---

6. Quantified Non-Functional Requirements

6.1 Response Time Analysis

Measurement Method: 10 iterations per endpoint, statistical analysis

```
Health Check Response Time:
├── Mean: 4.6ms
├── Median: 4.6ms
├── P95: 4.9ms
├── P99: 4.9ms
└── Standard Deviation: 0.2ms
```

Performance Classification:

- **Excellent** (<10ms): Health check
- **Good** (10-100ms): Assignments endpoint
- **Acceptable** (100-500ms): Issues endpoint
- **Slow but Functional** (>1s): AI operations

6.2 Throughput Analysis

Measurement Method: 15-second sustained load testing

```
Throughput Characteristics:
├── Simple Operations: 201.1 RPS
├── Database Operations: 11.8 RPS
├── Concurrent Users Supported: ~50
└── Request Success Rate: 100%
```

Scalability Assessment:

- **Current Capacity**: Adequate for prototype
- **Bottleneck**: Database query optimization
- **Scaling Strategy**: Read replicas, connection pooling

---

7. Architectural Decision Records

7.1 Decision: FastAPI over Express.js

Context: Backend framework selection
Options: FastAPI (Python) vs Express.js (Node.js)
Decision: FastAPI
Consequences:

- Better Python ecosystem for ML/AI

- Automatic API documentation
- Smaller ecosystem compared to Node.js

7.2 Decision: MongoDB over PostgreSQL

Context: Database selection
Options: MongoDB (NoSQL) vs PostgreSQL (SQL)
Decision: MongoDB
Consequences:

- Flexible schema for varied data types
- Good integration with Python
- Less powerful querying capabilities

7.3 Decision: External AI vs. Local Models

Context: AI implementation approach
Options: Gemini API vs Local ML models
Decision: Gemini API
Consequences:

- No ML infrastructure maintenance
- State-of-the-art models
- External dependency costs
- API rate limiting

Contributions:

- Fetching & Duplicate Detection - Krishna Koushik
- Contributor Data Fetch & Summarize & LLM Assignment & Storage - Vivek, Chandana
- Dashboard Display - Kriti
- Pub-Sub & SSE - Jahnavi
- Documentation & Testing - All

Reflections on the Project Process

The development of the **Intelligent Bug Report Management System** followed an iterative and architecture-centric approach. Key reflections include:

1. **Iterative Refinement**: We began with simple data-fetching scripts and evolved them into a comprehensive service layer. This allowed us to validate the GitHub API integration before building the complex AI-driven assignment logic.
2. **Architecture-Driven Design**: By establishing C4 models (Context, Container, Component) early, we maintained a clear understanding of system boundaries. This reduced integration friction when combining the FastAPI frontend with the Python backend services.
3. **Effective Documentation**: The use of Architecture Decision Records (ADRs) was instrumental in ensuring that technical choices (like selecting MongoDB Atlas over a relational database) were documented with clear rationale, preventing "decision drift" during the implementation phase.

Lessons Learned

1. **Challenges of Real-Time Sync**: Integrating with external APIs like GitHub taught us the importance of handling rate limits and network latency gracefully. Moving towards an event-driven synchronization model was a necessary response to these challenges.
2. **Flexibility in AI Integration**: We learned that LLM APIs can be volatile. Implementing the Strategy Pattern for the Gemini API integration proved to be a critical architectural decision, as it allows us to switch or update AI models without touching the core business logic.
3. **Data Schema Flexibility**: Building a system for dynamic bug reports reinforced the benefits of NoSQL databases. MongoDB Atlas allowed us to store varying issue formats and contributor metadata without the rigid constraints of a traditional schema, accelerating our development cycle.